Burak Demirci

2201292

COP4493 - AWS Academy Cloud Computing Fundamentals

Project-1

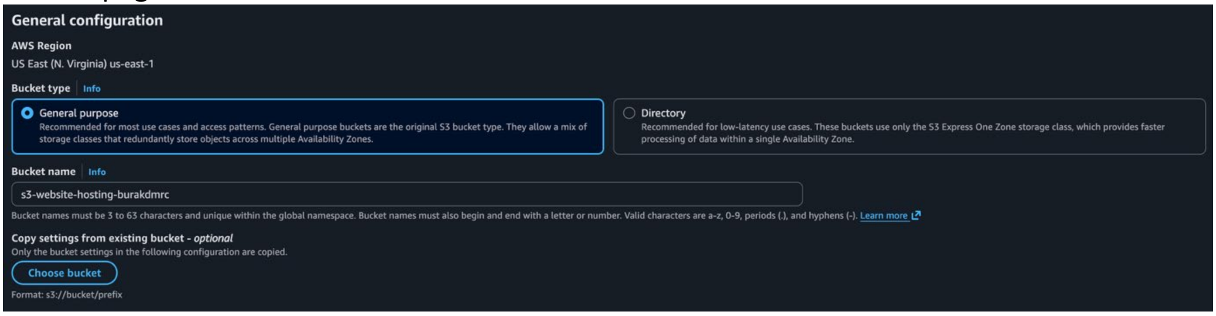
URL of Website: lb-website-hosting-burakdmrc-1713133902.us-east-1.elb.amazonaws.com

AWS Account ID: 895084669974

GitHub Repository URL: https://github.com/burakdemirci03/cop4493_website

S3 Bucket Creation

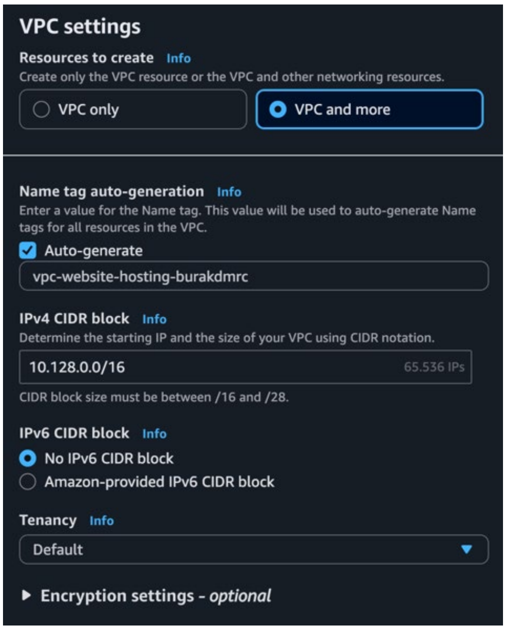
1. **Bucket Type**: Select **“General Purpose”**.
2. **Bucket Name**: Any name to identify the bucket, I gave **“s3-website-hosting-burakdmrc”**.
3. Other configurations can stay same, then proceed with “Create Bucket” at the end of the page.



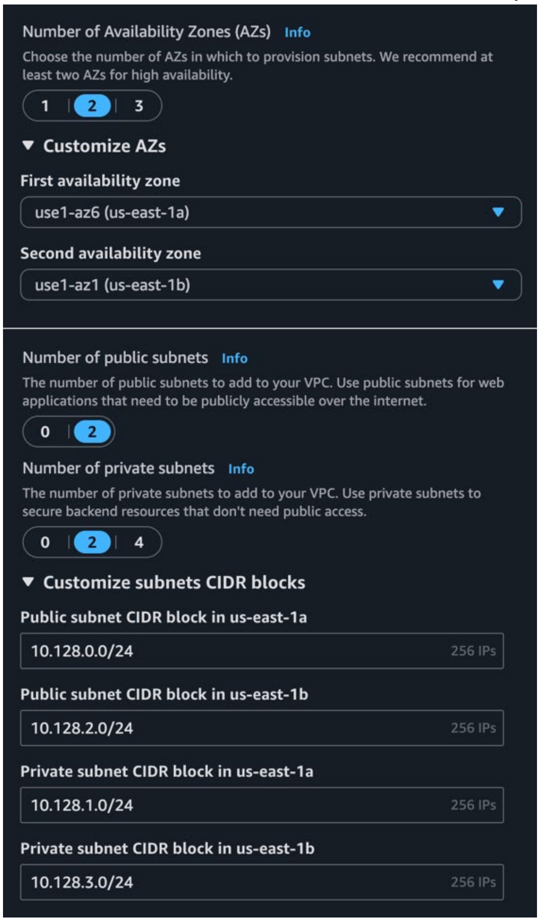
4. After creating the bucket, select the bucket and then upload the HTML, CSS and JS files using top left button.
5. Then go to “Properties” from top bar and find the Static Website Hosting in the bottom of the page. And click to “Edit” button.
6. **Static website hosting**: **Enable** the radio button.
7. **Hosting Type**: Select **“Host a static website”** area.
8. **Index Document**: Set the same HTML file as the name. Mine was **“index.html”**, I set it so. If an error endpoint is given, it can also be named the same way.
9. Don’t need to change anymore, so “Save Changes”.

VPC Creation

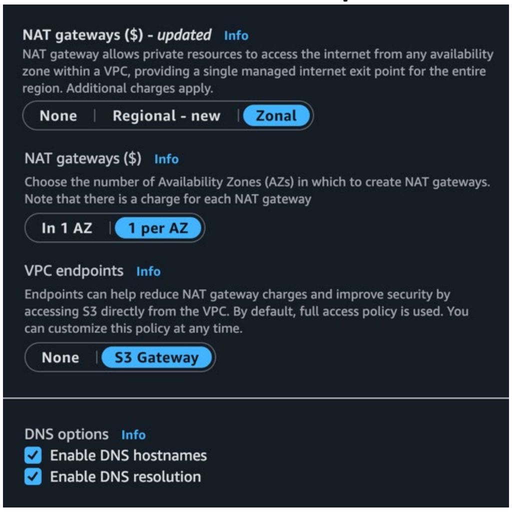
1. **Resources to Create**: Select **“VPC and more”**. So, while creating VPC, NAT and subnets are also created by AWS.
2. **Name Tag Auto-Generation**: **Enable “Auto-generate”** and name anyway you wish, I named myself **“vpc-website-hosting-burakdmrc”**.
3. **IPv4 CIDR Block**: I gave the address space as **“10.128.0.0/16”**. But any other suitable spaces can be given.
4. **IPv6 CIDR Block**: Select **“No IPv6 CIDR Block”**.



5. **Number of Availability Zones (AZs)**: Select **2**. And choose two different AZs, I choose **us-east-1a** and **us-east-1b**.
6. **Number of Public/Private Subnets**: Select **2** for both public and private subnets.
7. **Subnets CIDR Blocks**: Is optional to modify. I named;
    a. **Public Subnet CIDR Block in us-east-1a**: 10.128.0.0/24
    b. **Public Subnet CIDR Block in us-east-1b**: 10.128.2.0/24
    c. **Private Subnet CIDR Block in us-east-1a**: 10.128.1.0/24
    d. **Private Subnet CIDR Block in us-east-1b**: 10.128.3.0/24



8. **NAT Gateways**: Select **“Zonal”** first and then **“1 per AZ”** in order.



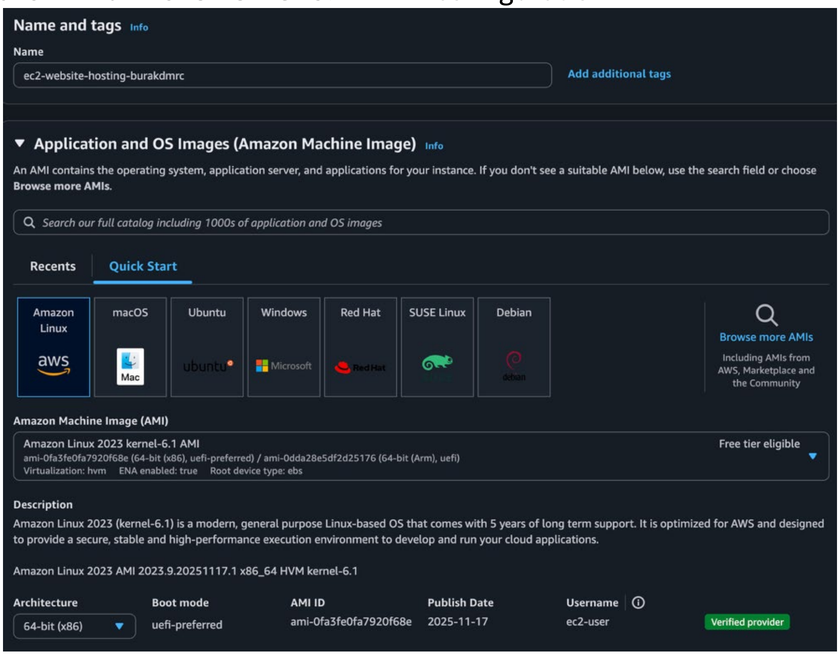
9. Then proceed with “Create VPC” finally.

<u>**PS**</u> If I do remember correctly, the **Subnets, Route Tables, Internet Gateway and NAT Gateways** are also configured properly while creating the VPC.
If not configured automatically;

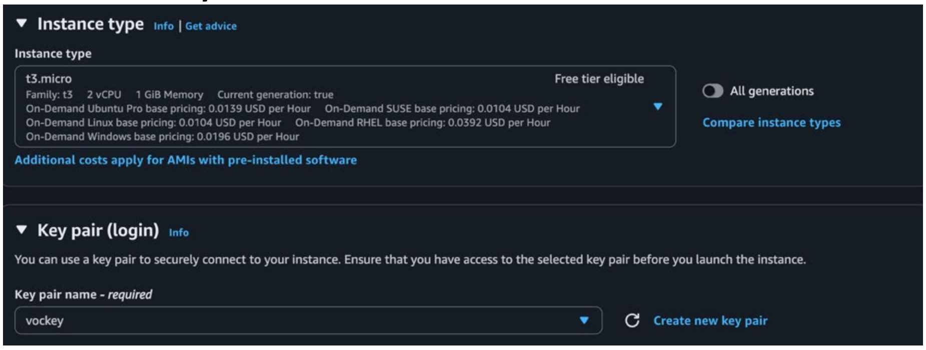
- **for Route Table for Public Subnet**: Any IPv4 address should target to the Internet Gateway *(0.0.0.0/0 -> igw-xxxxxx)*.
- **for Route Tables for Private Subnets**: Any IPv4 address should target to its corresponding NAT Gateway *(0.0.0.0/0 -> nat-xxxxxx)*.

EC2 Instance Creation

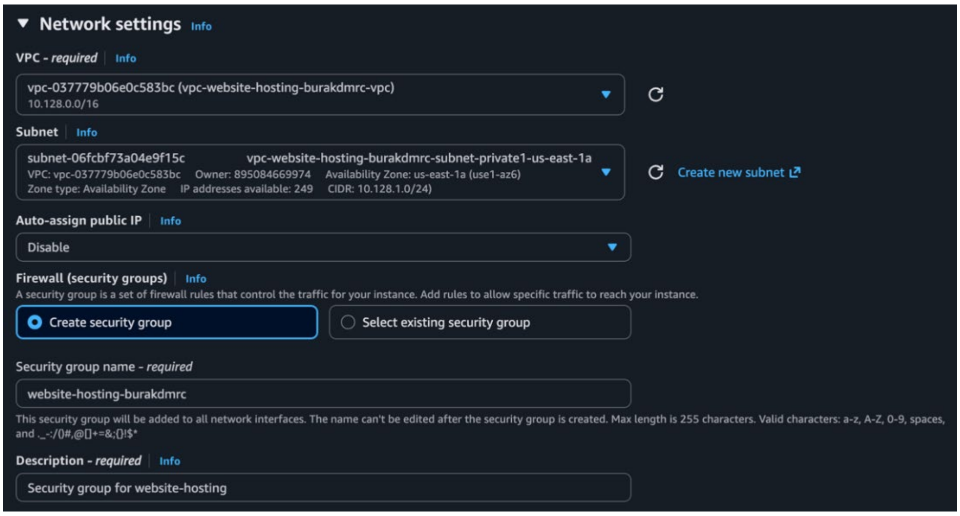
1. **Name**: Select any name you prefer, I named **“ec2-website-hosting-burakdmrc”**.
2. **Amazon Machine Image (AMI)**: Do not need to change the default **“Amazon Linux”** and **“Amazon Linux 2023 kernel-6.1 AMI”** configuration.



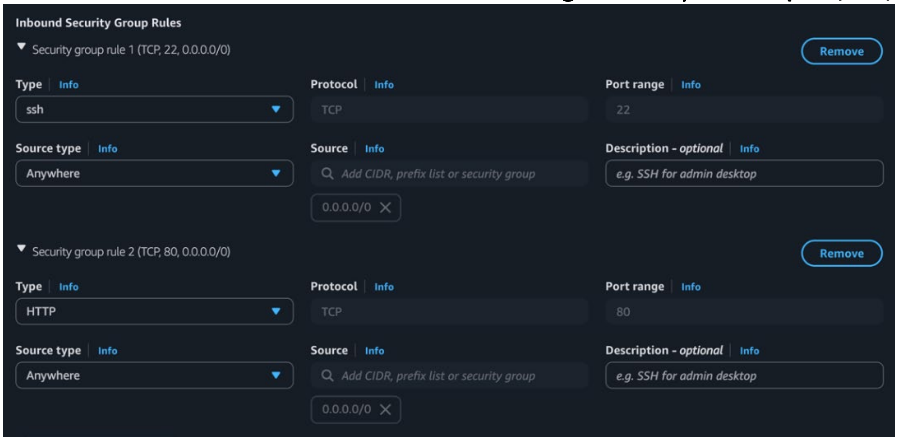
3. **Instance Type**: The default **t3.micro** can also remain the same.
4. **Key Pair**: Select **"vockey"**.



5. **VPC**: Select the recently created VPC, "**vpc-website-hosting-burakdmrc-vpc**".
6. **Subnet**: Choose any private subnet of given VPC, I choose us-east-1a private subnet "**vpc-website-hosting-burakdmrc-subnet-private1-us-east-1a**".
7. **Auto-assign Public IP**: **Disable** the setting.
8. **Firewall (Security Group)**: We do not have any security group yet so choose **"Create Security Group"**.
9. **Security Group Name**: Name any way you prefer, I named **"website-hosting-burakdmrc"**.



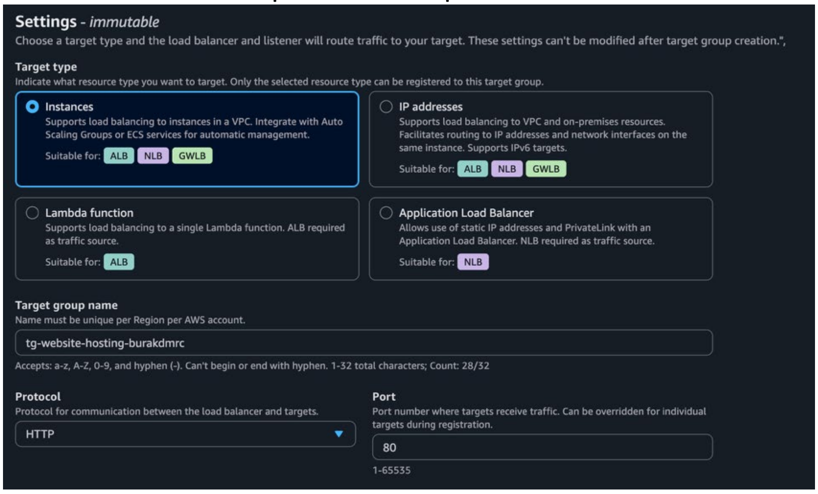
10. **Inbound Security Group Rules**: Keep the default ssh rule **(TCP, 22, 0.0.0.0/0)** and add a new rule that takes the HTTP connection and target to Anywhere **(TCP, 80, 0.0.0.0/0)**



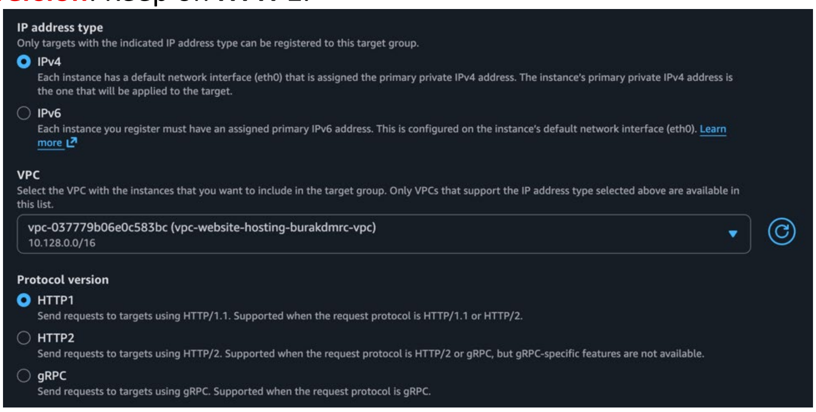
11. Then launch the EC2 instance.

12. We created this instance on only one subnet so, when created the first EC2 instance, select the created instance (given **“ec2-website-hosting-burakdmrc”**) and in top right bar select “Actions” and “Images and Templates”, then click **“Launch more like this”**. Then in opened tab;
    - **Subnet**: Change the subnet to the other private subnet. I choose private1-us-east-1a in first instance, so in this instance I changed to **”vpc-website-hosting-burakdmrc-subnet-private2-us-east-1b”**.
      (I did not launch another instance, which would cost too much in long-term)

13. In left scrollable are click **“Target Groups”**, then in top right click **“Create Target Group”**. We will need a target group in Load Balancer.
14. **Target Type**: Keep the setting in **Instances**.
15. **Target Group Name**: Give a unique name for Target Group, I gave **“tg-website-hosting-burakdmrc”**.
16. **Protocol & Port**: Choose **HTTP** protocol and port **80**.

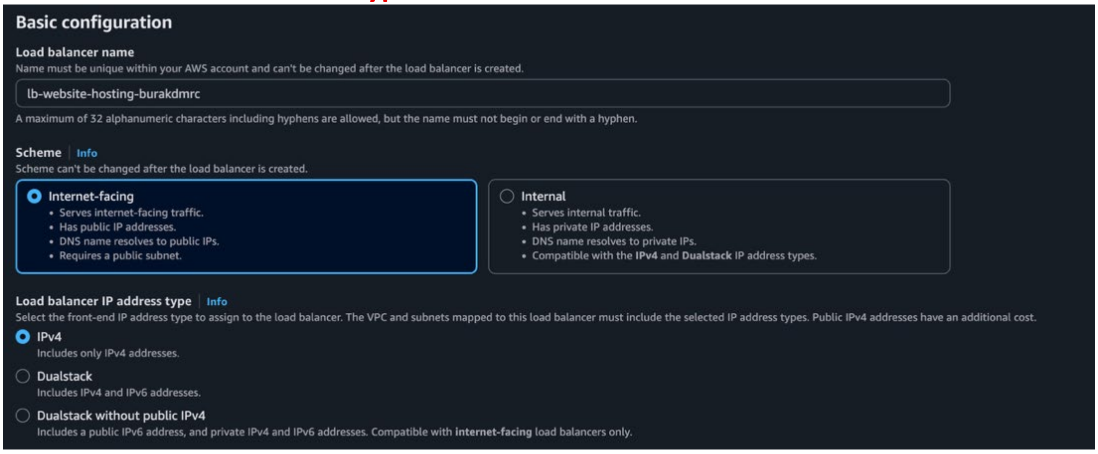


17. **IP Address Type**: Keep in **IPv4**.
18. **VPC**: Select the previous VPC, “**vpc-website-hosting-burakdmrc-vpc**”.
19. **Protocol Version**: Keep on **HTTP1**.

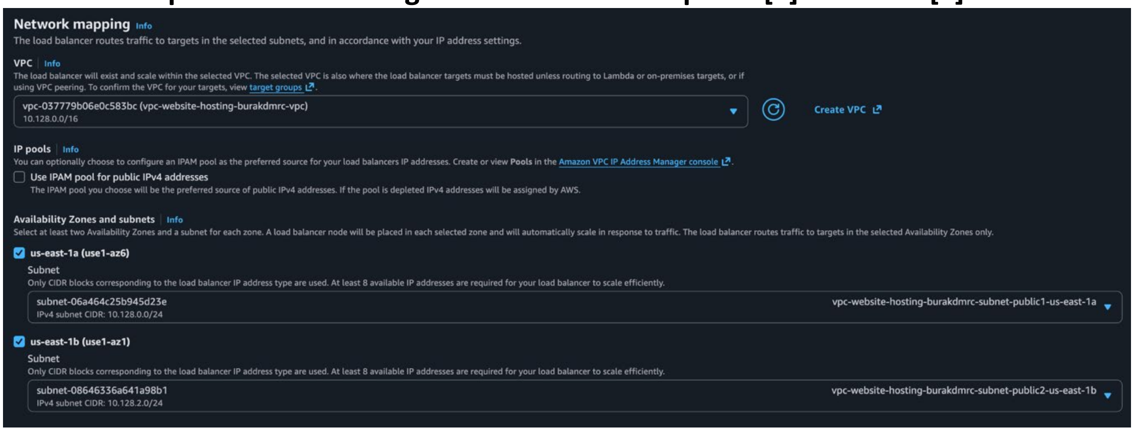


20. Then proceed to “Next” page.
21. **Register Targets**: Select the recently created EC2 instance, which was **“ec2-website-hosting-burakdmrc”** for me. Then click “Include as pending below” button and proceed to “Next” page where you would end up in “Create Target Group”.

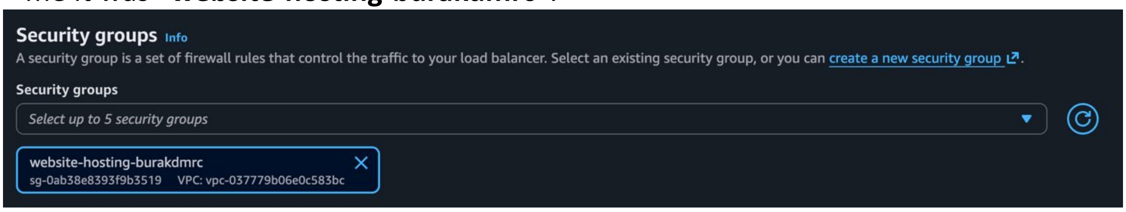
22. In left scrollable are click **“Load Balancer”**, then in top right click **“Create Load Balancer”**.
23. Select **Create** from **Application Load Balancer** section in **Load Balancer Types**.
24. **Load Balancer Name**: Give a unique name for Load Balancer, I gave **“lb-website-hosting-burakdmrc”**.
25. **Scheme**: Select **Internet-facing**.
26. **Load Balancer IP Address Type**: Choose **IPv4**.



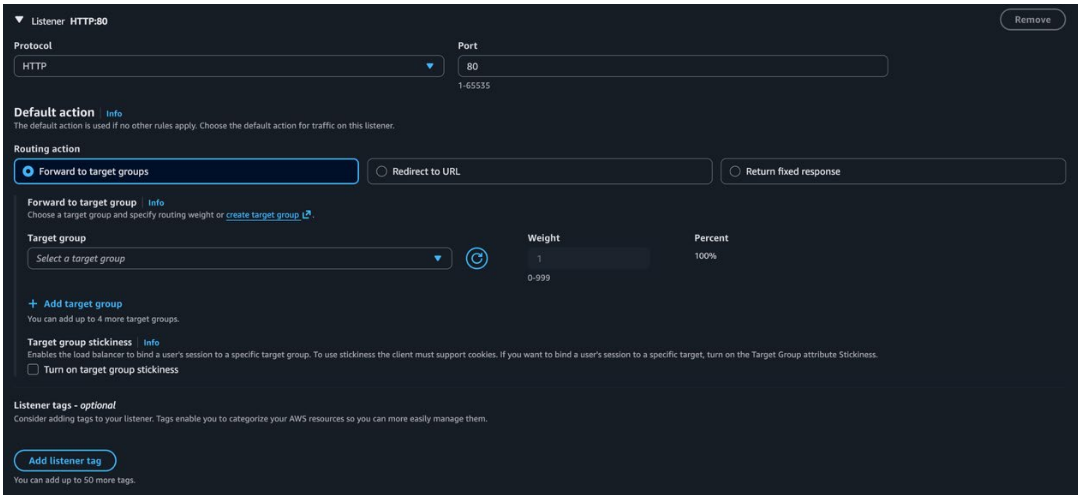
27. **VPC**: Select the previous VPC, “**vpc-website-hosting-burakdmrc-vpc**”.
28. **Availability Zones and Subnet**: Select both subnets and its corresponding public subnets **”vpc-website-hosting-burakdmrc-subnet-public[x]-us-east-1[x]”**.



29. **Security Groups**: Select the security group, which was created while creating VPC, for me it was **“website-hosting-burakdmrc”**.



30. **Protocol & Port**: Choose **HTTP** protocol and port **80**.
31. **Routing Action**: Select **“Forward to target groups”**.
32. **Target Group**: Select the recently created target group, **“tg-website-hosting-burakdmrc”**.
    (Since I have used target group before, I cannot choose it while showing in another instance)



33. Then continue with “Create Load Balancer” which is located the end of page.

Web Server Configuration

1. Select the EC2 instance and click to the “Connect” button at the top-right corner.
2. In instance connection try one of the following;
    a. Go back to the instances page and select the instance, then proceed with Actions > Networking > Manage IP Address. Open up the collapsable Network Interface section (named like, “eth0: eni-[xxxx] - 10.128.1.0/24”) and turn on the “Auto-assign Public IP” option and Save. Then go back to Connection tab, select **“EC2 Instance Connect”** and “Connect using a Public IP”.
    (turn off the “Auto-assign Public IP” when all configuration are done)
    b. Connect with **“SSH Client”** using the instruction in readme manual on AWS Academy website.
    c. If possible, connect using **“Session Manager”**.
    (while doing project, most of time this option was not open for me but now it is open, even if I do not know why)
3. Create the Apache Server using following instructions;

```
sudo yum update
sudo yum install httpd
sudo systemctl start httpd
sudo systemctl enable httpd
```

4. Then load the objects -HTML, CSS and JS files- from S3 to the Apache server using;

```
sudo aws s3 sync s3://s3-website-hosting-burakdmrc /var/www/html
```

Website should be accessible by the DNS record of the load balancer now!